

SIMATS ENGINEERING
C01 - ASSESSMENT TOOL 2

NAME: Girija B
REG NO: 192311044
COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1103

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

*I Girija B/192311044 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

GIRIJA B

1. Object-Oriented Design in Smart Healthcare Systems

A Smart Healthcare System manages patients, doctors, appointments, and medical records.

Explain how object-oriented concepts such as class, object, encapsulation, inheritance, and polymorphism are applied in this system. Analyze how these concepts improve scalability, security, and maintainability.

Object-Oriented Programming (OOP) is widely used in Smart Healthcare Systems because it models real-world healthcare entities as objects, making the software organized, reusable, and easy to manage.

Application of OOP Concepts

- **Class:** A class acts as a blueprint for creating objects. Examples include **Patient, Doctor, Appointment, MedicalRecord, Prescription, and Admin**.
- **Object:** An object is an instance of a class. Each patient, doctor, or appointment in the system is represented as a separate object with its own data and behavior.
- **Encapsulation:** Sensitive information such as patient medical history, diagnosis, and prescriptions is kept private and accessed only through getter and setter methods, ensuring data security and confidentiality.
- **Inheritance:** A common **Person** class contains shared attributes like **name, age, contact number, and address**. Classes such as **Patient** and **Doctor** inherit these properties, reducing code duplication and improving reusability.
- **Polymorphism:** The same method can perform different operations depending on the object. For example, the **consult()** method can provide **online consultation, offline consultation, or emergency consultation** based on the appointment type.

Benefits of OOP in Smart Healthcare Systems

- **Scalability:** New modules such as **Pharmacy, Laboratory, Billing, Insurance, and Telemedicine** can be added without affecting existing modules.
- **Security:** Encapsulation protects confidential patient records and allows only authorized doctors or administrators to access sensitive information.
- **Maintainability:** The modular structure makes debugging, testing, and updating the software easier. Changes in one module do not significantly affect others.
- **Code Reusability:** Inheritance reduces repeated code, saving development time and improving software quality.
- **Flexibility:** Polymorphism allows the system to support multiple healthcare services with minimal code changes.
- **Reliability:** Objects represent real-world entities accurately, making the system easier to understand and reducing software errors.

Conclusion

Object-Oriented Programming provides a structured approach to developing Smart Healthcare Systems. By using **classes, objects, encapsulation, inheritance, and polymorphism**, the system becomes **secure, scalable, flexible, reusable, and easy to maintain**, resulting in efficient healthcare management and high-quality software.

2. Applying Encapsulation and Abstraction in Online Banking

An Online Banking System provides services such as fund transfer, account management, and transaction tracking.

Discuss the concepts of abstraction and encapsulation in object-oriented design. Compare their roles and analyze how they help in protecting sensitive data and simplifying system complexity.

An **Online Banking System** uses Object-Oriented Programming (OOP) to securely manage services such as **fund transfer, account management, balance inquiry, and transaction tracking**. Two important OOP concepts used are **encapsulation** and **abstraction**.

Encapsulation

Encapsulation means **binding data and methods together** and restricting direct access to sensitive information.

- Account details and PIN are stored as **private** data.
- Access is provided only through methods like **deposit(), withdraw(), and checkBalance()**.
- Prevents unauthorized access and protects customer information.

Abstraction

Abstraction means **showing only essential features while hiding internal implementation details**.

- Customers can perform operations like **fund transfer** or **balance check** without knowing how the banking server processes the transaction.
- It simplifies the system by exposing only the required functionalities.

Comparison

Encapsulation	Abstraction
Protects data from unauthorized access	Hides implementation details
Achieves data security	Reduces system complexity
Implemented using private variables and getter/setter methods	Implemented using abstract classes and interfaces

Benefits in Online Banking

- **Data Security:** Protects account numbers, passwords, and transaction details.
- **Simplified Usage:** Customers interact with simple banking services without understanding the internal processing.
- **Maintainability:** Internal changes can be made without affecting users.
- **Reliability:** Ensures secure and efficient banking operations.

Conclusion

Encapsulation secures sensitive banking data, while **abstraction** simplifies system usage by hiding complex processes. Together, they make the Online Banking System **secure, user-friendly, maintainable, and reliable**.

3. Modeling Object Relationships in a University Management System

A University Management System manages students, faculty, departments, and courses. Analyze different object relationships such as association, aggregation, and composition. Explain how appropriate relationship modeling improves system flexibility, reusability, and maintainability.

A **University Management System** manages **students, faculty, departments, and courses**. Object relationships such as **association, aggregation, and composition** help organize these entities efficiently.

Object Relationships

- **Association:** Represents a relationship where two objects are connected but exist independently.
Example: A **Student** enrolls in a **Course**, and a course can have many students.
- **Aggregation:** Represents a "has-a" relationship where one object contains another, but both can exist independently.

Example: A **Department** has **Faculty Members**. Even if a department is removed, faculty members can still exist.

- **Composition:** Represents a **strong "part-of"** relationship where one object cannot exist without the other.

Example: A **Student** has a **Student Record**. If the student is deleted, the student record is also deleted.

Benefits of Relationship Modeling

- **Flexibility:** New departments, courses, or faculty can be added easily.
- **Reusability:** Common objects can be shared across different modules, reducing code duplication.
- **Maintainability:** Clear relationships make the system easier to understand, update, and debug.
- **Modularity:** Each class performs a specific role, improving software organization.

Conclusion

Using **association, aggregation, and composition** creates a **structured, modular, and efficient** University Management System. These relationships improve **flexibility, code reusability, maintainability, and scalability**, resulting in high-quality software.

4. Importance of SDLC in E-Commerce Development

An E-Commerce Platform is developed to support product management, online payments, and order tracking.

Explain the phases of Software Development Life Cycle (SDLC). Analyze how each phase contributes to software quality, reliability, and customer satisfaction.

An **E-Commerce Platform** supports **product management, online payments, order tracking, and customer accounts**. The **Software Development Life Cycle (SDLC)** provides a structured process to develop high-quality software.

Phases of SDLC

1. **Requirement Analysis:** Gather customer requirements such as product catalog, payment gateway, shopping cart, and order tracking.
2. **System Design:** Design the system architecture, database, user interface, and security features.
3. **Implementation (Coding):** Develop modules like login, product management, payment, and order processing.
4. **Testing:** Perform unit, integration, system, and user acceptance testing to identify and fix errors.
5. **Deployment:** Release the application for customers to use.
6. **Maintenance:** Update the software, fix bugs, improve performance, and add new features.

Contribution of Each Phase

- **Software Quality:** Proper design and thorough testing reduce errors and improve performance.
- **Reliability:** Testing ensures secure payments, accurate order processing, and stable system operation.
- **Customer Satisfaction:** A user-friendly interface, fast performance, and regular maintenance improve the customer experience.
- **Security:** Requirement analysis and testing help implement secure payment processing and protect customer data.

Conclusion

Following the **SDLC** ensures the E-Commerce Platform is **high-quality, reliable, secure, and easy to maintain**, leading to better customer satisfaction and successful software development.

5. Selecting an Appropriate SDLC Model for Mobile App Development

A company develops a Mobile Learning Application where customer requirements change frequently.

Compare Agile, Waterfall, and Spiral models. Analyze their advantages and limitations and recommend the most suitable SDLC model with justification.

A **Mobile Learning Application** requires frequent updates because customer requirements change regularly. Choosing the right **SDLC model** is essential for successful development.

Comparison of SDLC Models

1. Waterfall Model

- Development follows a **sequential** process.
- Suitable when requirements are fixed.
- **Advantages:** Simple, well-documented, easy to manage.
- **Limitations:** Difficult to make changes after development begins.

2. Agile Model

- Development is done in **small iterations (sprints)** with continuous customer feedback.
- **Advantages:** Flexible, quick delivery, easy to handle changing requirements.
- **Limitations:** Requires active customer involvement and experienced teams.

3. Spiral Model

- Combines **iterative development** with **risk analysis**.
- Suitable for large and high-risk projects.
- **Advantages:** Better risk management, supports requirement changes.
- **Limitations:** Expensive, complex, and time-consuming.

Comparison Table

Model	Advantages	Limitations
Waterfall	Simple, easy to manage	Not suitable for changing requirements
Agile	Flexible, fast delivery, continuous feedback	Requires frequent customer involvement
Spiral	Excellent risk management	Costly and complex

Recommended Model: Agile

The **Agile Model** is the most suitable because:

- It supports **frequent requirement changes**.
- Delivers working software in short iterations.
- Encourages continuous customer feedback.
- Improves software quality through regular testing.
- Enables faster updates and feature enhancements.

Conclusion

For a **Mobile Learning Application**, the **Agile SDLC model** is the best choice because it provides **flexibility, faster development, continuous improvement, and high customer satisfaction**, making it ideal for projects with changing requirements.

6. Modular Design in a Hospital Management System

A Hospital Management System experiences difficulties due to tightly coupled modules and frequent maintenance issues.

Analyze the existing problems and redesign the system using object-oriented principles. Explain how modular design improves maintainability, scalability, and code reusability.

A **Hospital Management System** may face problems such as **tightly coupled modules, difficult maintenance, and code duplication**. Applying **Object-Oriented Programming (OOP)** principles helps redesign the system into independent and reusable modules.

Existing Problems

- Tightly coupled modules make changes difficult.
- A change in one module affects other modules.
- Frequent maintenance increases time and cost.
- Code duplication reduces efficiency and increases errors.

Redesign Using OOP Principles

- Create separate **classes** for **Patient, Doctor, Appointment, Billing, Pharmacy, and Medical Record**.
- Use **Encapsulation** to protect patient information.
- Use **Inheritance** to reuse common features through a **Person** class.
- Use **Polymorphism** to support different consultation or billing methods.
- Keep each module independent with well-defined responsibilities.

Benefits of Modular Design

- **Maintainability:** Easy to update or fix one module without affecting others.
- **Scalability:** New modules like Laboratory or Insurance can be added easily.
- **Code Reusability:** Common functionalities are reused, reducing code duplication.
- **Flexibility:** Independent modules simplify testing and future enhancements.
- **Reliability:** Modular structure reduces errors and improves software quality.

Conclusion

Using **object-oriented principles** and **modular design** makes the Hospital Management System **maintainable, scalable, reusable, and reliable**, resulting in efficient healthcare software development.

7. Enhancing Software Quality Using Design Patterns

A Hotel Reservation System suffers from duplicated code and inefficient object creation mechanisms.

Analyze the system issues and apply suitable design patterns such as Factory Pattern, Singleton Pattern, or Observer Pattern. Explain how these patterns improve flexibility, maintainability, and software quality.

A **Hotel Reservation System** may face issues such as **duplicate code, inefficient object creation, and difficult maintenance**. These problems can be solved using **design patterns** like **Factory, Singleton, and Observer**.

System Issues

- Duplicate code increases maintenance effort.
- Creating multiple objects reduces performance.
- Changes in one module affect other modules.
- Difficult to manage notifications and updates.

Applying Design Patterns

- **Factory Pattern:** Creates objects without exposing the object creation process.
Example: Creates different room types such as **Standard Room, Deluxe Room, and Suite**.
- **Singleton Pattern:** Ensures only one instance of a class exists.
Example: A single **Database Connection** or **Booking Manager** is shared throughout the application.
- **Observer Pattern:** Automatically notifies users when changes occur.
Example: Customers receive notifications when a booking is confirmed, cancelled, or modified.

Benefits of Design Patterns

- **Flexibility:** Easy to add new room types or booking features.
- **Maintainability:** Reduces code duplication and simplifies updates.
- **Code Reusability:** Reusable design reduces development time.
- **Software Quality:** Improves performance, reliability, and scalability.
- **Loose Coupling:** Modules work independently, making the system easier to modify.

Conclusion

Using **Factory, Singleton, and Observer Patterns** improves the Hotel Reservation System by making it **flexible, maintainable, reusable, scalable, and efficient**, resulting in better software quality and easier maintenance.

8. UML-Based Modeling for an Online Examination System

An Online Examination System manages students, instructors, examinations, and result processing.

Design UML diagrams such as:

- **Use Case Diagram**
- **Class Diagram**
- **Activity Diagram**

Analyze how UML modeling helps in system visualization, requirement analysis, and stakeholder communication.

An **Online Examination System** manages **students, instructors, examinations, question papers, and results**. UML (Unified Modeling Language) helps visualize and design the system before implementation.

1. Use Case Diagram

Shows how users interact with the system.

- **Student:** Login, Take Exam, View Result
- **Instructor:** Create Exam, Upload Questions, Evaluate Answers
- **Admin:** Manage Users, Schedule Exams, Generate Reports

2. Class Diagram

Represents the classes and their relationships.

- **Student**
- **Instructor**
- **Exam**
- **Question**
- **Result**

Relationships:

- Student → Takes → Exam
- Instructor → Creates → Exam

- Exam → Contains → Questions
- Exam → Generates → Result

3. Activity Diagram

Shows the workflow of the examination process.

Login → Select Exam → Start Exam → Submit Answers → Evaluation → Display Result

Benefits of UML Modeling

- **System Visualization:** Provides a clear graphical representation of the system.
- **Requirement Analysis:** Helps identify system requirements before coding.
- **Stakeholder Communication:** Improves communication between developers, clients, and users.
- **Maintainability:** Makes future modifications easier by providing clear documentation.
- **Reduced Errors:** Detects design issues early, reducing development cost.

Conclusion

UML diagrams such as **Use Case, Class, and Activity Diagrams** help develop a **well-structured, maintainable, and reliable** Online Examination System by improving **system visualization, requirement analysis, and stakeholder communication**.